Django Day 5: Handling User Input (Forms)

Topic: Django ModelForms & POST Requests

Trainer: Saurabh Sir

Focus: Logic, Syntax & Security (CSRF)

1. Introduction: The Problem & Solution

Until now, only the Admin (Superuser) could add data. But normal users (like on Facebook or Instagram) need a way to fill out forms.

The Django Solution (ModelForm):
Instead of writing long HTML code (`<input type="text">`), we let Django read our `models.py` file. Django will automatically create a Form that matches your Database table perfectly.

2. Step-by-Step Implementation

Step 1: Create `forms.py`

By default, Django doesn't have a `forms.py` file. You must create it inside your app folder.

```
from django import forms
from .models import Student  # Import the Model

class StudentForm(forms.ModelForm):
    class Meta:
```

```
        model = Student
        fields = ['name', 'age', 'address']
```

Code Explanation:

- `forms.ModelForm` - This tells Django: "Build a form based on a database model."
- `class Meta:` - This is the configuration area.
- `model = Student` - Tells Django which table to connect to.
- `fields = [...]` - Tells Django which columns to show in the form. We didn't include IDs because those are auto-generated.

Step 2: Logic in `views.py`

This View is special because it handles two jobs at once: **Showing** the empty form and **Saving** the filled data.

GET vs. POST:
1. **GET:** When the user just opens the page url (displays empty form).
2. **POST:** When the user clicks the "Submit" button (sends data to server).

```python
from django.shortcuts import render, redirect
from .forms import StudentForm

def add_student(request):
    # Check if the user clicked the Submit button
    if request.method == 'POST':
        form = StudentForm(request.POST) # Fill form with user data
        if form.is_valid():
            form.save()          # Save to Database
            return redirect('home') # Go back to home page

    # If the user is just opening the page (GET request)
    else:
        form = StudentForm() # Create an empty form

    return render(request, 'myapp/add.html', {'form': form})
```

> ⚠️ **Logic & Case Sensitivity Alert:**
>
> - `request.POST` - Must be capital letters. `request.post` will give an error.
> - **Indentation:** The `else` block must align with the `if` block. If the indentation is wrong, the form might never show up.
> - `redirect` - Don't forget to import this at the top!

Step 3: The Template (add.html)

Create a new file `add.html`. We use a special tag here for security.

```
<!DOCTYPE html>
<html>
<body>
    <h1>Add New Student</h1>

    <!-- method="POST" is mandatory for sending data -->
    <form method="POST">

        <!-- Security Token (Required by Django) -->
        {% csrf_token %}

        <!-- This prints the inputs automatically -->
        {{ form.as_p }}

        <button type="submit">Save Data</button>
    </form>
</body>
</html>
```

Detailed Explanation:

- `method="POST"` - This tells the browser to send data hiddenly (securely), not in the URL bar.
- `{% csrf_token %}` - **(Very Important)** Cross-Site Request Forgery token. Think of this as a security badge. If you forget this, Django will reject the form with a **403 Forbidden Error**.

- `{{ form.as_p }}` - "as_p" means "As Paragraph". Django takes the form fields defined in Python and converts them into HTML inputs wrapped in <p> tags.

Step 4: Add URL (urls.py)

```
from . import views

urlpatterns = [
    # ... previous urls ...
    path('add/', views.add_student, name='add_student'),
]
```